

实验六 图的综合应用

二、实验内容

对给定的输入内容，完成实验任务

输入顶点集: 1 2 3 4 5 6 7 8

输入边的集合: 1 2

1 3

24

25

48

58

3 6

37

6 7

- (1) 创建一个图（可用邻接矩阵或邻接表的方式进行存储）；
- (2) 输入选项：0 或 1，0 为 DFS，1 为 BFS
- (3) 分别输出 DFS 和 BFS 两种遍历序列。

[illegible]

```

290 int main(){
291     int choose,n,v=0;
292     A: menu1();
293     cout<<"请选择操作: ";
294     cin>>choose;
295     if(choose==1){/*邻接矩阵*/
296         cout<<"|---图将以邻接矩阵进行存储! ---|"<<endl;
297         AMGraph G;
298         CreatUDNAM(G);
299         do{
300             menu3();
301             cout<<"请选择操作: ";
302             cin>>n;
303             switch(n){
304                 case 0:
305                     cout<<"该图的深度优先遍历结果为: ";
306                     DFS_AM(G,v);
307                     cout<<endl;
308                     break;
309                 case 1:
310                     cout<<"该图的广度优先遍历结果为: ";
311                     BFS_AM(G,v);
312                     break;
313                 case 2:
314                     cout<<"该图的邻接矩阵为: "<<endl;
315                     putAMGrapp(G);
316                     break;
317                 case 3: goto A;
318                 case 4: return 0;
319             }
320         }while(n!=3);
321     }
322 }
323 else if(choose==2){/*邻接表*/
324     cout<<"|----图将以邻接表进行存储! ----|"<<endl;
325     ALGraph g;
326     CreatUDNAL(g);
327     do{
328         menu2();
329         cout<<"请选择操作: ";
330         cin>>n;
331         switch(n){
332             case 0:
333                 cout<<"该图的深度优先遍历结果为: ";
334                 DFS_AL(g,v);
335                 cout<<endl;
336                 break;
337             case 1:
338                 cout<<"该图的广度优先遍历结果为: ";
339                 BFS_AL(g,v);
340                 break;
341             case 2:
342                 cout<<"该图的邻接表为: "<<endl;
343                 printf_ALGraph(g);
344                 break;
345             case 3: goto A;
346             case 4: return 0;
347         }
348     }while(n!=3);
349 }
350 }
351 else if(!choose){
352     cout<<"欢迎下次使用! "<<endl;
353     return 0;
354 }
355 return 0;
356 }
357
358

```

```

182 void CreatUDNAL(ALGraph &g){
183     int v1,v2,i,j;
184     ArcNode *p1,*p2;
185
186     cout<<"|---请输入总顶点数与总边数! ---|"<<endl;
187     cin>>g.vexnum>>g.arcnum;//输入总点数与总边数
188     cout<<"|-----请输入顶点值! -----|"<<endl;
189     for(int i=0;i<g.vexnum;++i){
190         cin>>g.vertices[i].data;//输入点值
191         g.vertices[i].firstarc=NULL;//表头结点的指针域初始化为空
192     }
193     cout<<"|-----请输入边的值! -----|"<<endl;
194     for(int k=0;k<g.arcnum;++k){
195         cin>>v1>>v2;//输入边的两个顶点
196         i=LocateVex2(g,v1);//定位i
197         j=LocateVex2(g,v2);//定位j
198         p1=new ArcNode;//生成边结点p1
199         p1->adivex=j;//p1的邻接点记为j
200         p1->nextarc=g.vertices[i].firstarc;
201         g.vertices[i].firstarc=p1;
202         /*将结点p1插入顶点vi的边表头部*/
203         p2=new ArcNode;//生成新的边结点p2
204         p2->adivex=i;//p2的邻接点记为i
205         p2->nextarc=g.vertices[j].firstarc;
206         g.vertices[j].firstarc=p2;
207         /*将结点p2插入顶点vj的边表头部*/
208     }
209     cout<<"|-----以邻接表建立图完成! -----|"<<endl;
210 }

```

```

213 /*邻接表的深度优先遍历*/
214 void DFS_AL(ALGraph g,int v){
215     int w;
216     cout<<g.vertices[v].data<<" ";//输出第一个
217     visitDFS[v]=true;//标记
218     ArcNode *p;//指针
219     p=g.vertices[v].firstarc;//指向v的邻接点
220     while(p!=NULL){//结束标志为遍历完成
221         w=p->adivex;//找到v的下个点
222         if(visitDFS[w]!=true)//未被访问过的点
223             DFS_AL(g,w);//递归调用, 深度遍历
224         p=p->nextarc;
225     }
226 }
227 /*建立BFS的visit数组*/
228 bool visitBFS[MVNum];
229 /*邻接表的广度优先遍历*/
230 void BFS_AL(ALGraph &g,int v){
231     SqQueue q;
232     ArcNode *p;//指针指向邻接表
233     int w;
234     InitQueue(q,g.vexnum);//创建队列
235     cout<<g.vertices[v].data<<" ";//输出第一个
236     EnQueue(q,v);//将其入队
237     visitBFS[v]=true;//标志
238     while(!QueueEmpty(q)){//队列非空
239         w=DeQueue(q);//将元素出队
240         p=g.vertices[w].firstarc;//p指向其首结点
241         while(1){//当p的指向为空时跳出循环
242             if(visitBFS[p->adivex]!=true){//判断是否被遍历过
243                 cout<<g.vertices[p->adivex].data<<" ";//输出指针p指向
244                 visitBFS[p->adivex]=true;//标记
245                 EnQueue(q,p->adivex);//将指针p指向的值入队
246             }
247             if(p->nextarc==NULL){//若为最后一个结点则跳出该循环
248                 break;
249             }
250             else{
251                 p=p->nextarc;//不是最后一个结点则p指向下一个结点

```